PRESAGE — build status

Last updated: 21 July 2026 **Verified against:** a clean run of the full check suite on this commit.

This is the running record of what is built and what is left. **Update it when you finish something** — the person picking up next shouldn't have to reverse-engineer the state from the diff.

TL;DR

The **product is complete and demoable end to end.** All 7 screens work, the live-call pipeline runs, the analyzer scores artifacts, and every component has a working fallback. Nothing is a stub.

What's left is the **outer ring**: real audio in, real notifications out, real payment rails, and a bigger corpus so the trained model beats the regexes it currently loses to. All four are additive — none of them requires unpicking what's there.

Measure	Where it stands
Lines of first-party code	~12,400 (Python + TS/TSX)
Backend tests	**16 / 16 passing**
Contract check	**passing** (8 enums + version + 24-frame mock)
Frontend typecheck + build	**passing**, zero errors
CI	**green** on py3.9 + py3.12 + frontend, ~1 min
Runs on a clean clone	**yes** — no key, no GPU, no network

Done

Contract (`schema/`) — complete

- `models.py` (Pydantic) and `types.ts` (TS unions) as a single source of truth for all 8 enums: `Stage` (8), `ThreatLevel` (5), `VictimState` (7), `PaymentState` (5), `GuardianState` (4), `Verdict` (3), `EventKind` (10).
- `check_contract.py` fails the build on any Python↔TS drift. **Passing.**
- `CONTRACT_VERSION = 1`, asserted on both sides.
- `mock-stream.json` — 24 `StateFrame`s + 24 `Event`s. Validated by the contract check, and the reason the UI still renders with the API down.

Backend engine (`services/api/engine/`) — complete

Module	State	Notes
`classifier.py`	✅	Both backends + **promotion gate** on measured F1, not file presence
`threat.py`	✅	4-signal weighted fusion, returns named drivers, ratchets (fast up / slow down)
`twin.py`	✅	Fitted transition matrix + dwell times → time-to-payment forecast
`coercion.py`	✅	Victim-side stress, **independent** of the classifier. Text-only, capped lower
`passport.py`	✅	Mechanical identity checks, PASS/FAIL/**UNKNOWN**, each with a citation
`upi.py`	✅	VPA + QR structural checks, no blocklist, no network call
`analyzer.py`	✅	Stateless path — reuses the *same* engine as the live path
`session.py`	✅	State machine → idempotent `StateFrame` snapshots + one-shot `Event` edges

Thresholds live in one screen of `session.py` : guardian at **70**, payment hold at **55**.

API (`services/api/`) — complete

- **17 endpoints** across `routes/analyze.py` and `routes/session.py` , plus a WebSocket pushing frames at 4 Hz.
- `/api/health` reports live-vs-degraded per component. Startup warm-loads everything so the first click isn't a 3-second cold start.
- CORS locked to `localhost:5173` / `127.0.0.1:5173` .
- Upload path capped at 4 MB, accepts `.txt` / `.json` / `.csv` .

Retrieval + coach (`services/api/rag/` , `knowledge/`) — complete

- BM25 **and** dense (sentence-transformers) backends, auto-selecting with fallback. 26 chunks across 3 curated documents (RBI advisories, scam playbooks, UPI safety).
- `coach_library.json` — 14 human-reviewed lines, delivered **verbatim**. An LLM may rank or reword the *explanation*; it never writes the line and never touches a score.

Frontend (`apps/web/`) — complete

- All **7 routes** built: `/` , `/dashboard` , `/console` , `/guardian` , `/analyzer` , `/knowledge` , `/model` .
- `AppShell` , `CommandPalette` (`⌘K`), `RouteBoundary` , GSAP motion, a `three` WebGL `ThreatField` , light/dark theming.
- `useLiveSession` (WebSocket), `useHealth` , `useStreamPlayer` (mock replay).
- **Pure renderer** — zero threat maths, thresholds, or stage rules in React. Every number is a contract field. Typecheck clean.

ML pipeline (`ml/`) — complete and reproducible

- Full offline chain: `generate_calls.py` → `paraphrase.py` → `build_dataset.py` → `train.py` → `eval_backends.py` .
- **Corpus is committed** (320 calls; 931 train / 138 val / 171 test), so the checkpoint is regenerable rather than lost.
- Split is **leave-archetypes-out by call** — the strict one, which is what caught the memorisation (see below).
- Both LLM generation backends work (Gemini free tier, local Ollama).

Repo hygiene — done in this commit

- `.gitignore` excludes `.env`, venvs, `node_modules`, and the ~3.6 GB of model weights — while **keeping** `backend_comparison.json` and `metrics.json`, which are load-bearing.
- `.env.example` committed as the template. No secrets in history.
- Clone is **2.5 MB / 109 files**.
- `pytest` added to `requirements.txt` — it was missing, so the checks the README asks for could not be run on a clean clone.

CI — added, green

`.github/workflows/ci.yml` runs on every push and PR to `main`. **~1 minute, all jobs passing.**

Job	What it does
`backend (py3.9)`	install → 16 verdict regressions → contract check → boot API, assert `/api/health`
`backend (py3.12)`	same, on the other end of the claimed range
`frontend`	`npm ci` → typecheck → `vite build`

The health assertion is the one worth keeping: it fails if `transitions.json` goes missing, if the coach library empties, or if the knowledge base loads nothing — all of which otherwise surface as a blank panel mid-demo. The runner has no checkpoint, so it also asserts the documented clean-clone path (`lexical | no checkpoint exported`).

Pending

Ordered by how much they'd improve the product per hour spent.

P0 — Corpus expansion (unblocks the model)

This is the single highest-value task and it is squarely a data problem.

The MuRIL checkpoint trains beautifully and then *loses to a pile of regexes*:

backend	val macro-F1	test macro-F1 (held-out archetypes)
lexical baseline	—	**0.368**
fine-tuned MuRIL	0.983	**0.221**

The 0.98 is memorisation — 320 synthetic calls give an 8-way classifier enough surface detail to recognise the *archetype* rather than the *stage*. Per-class test F1 shows exactly where it collapses:

stage	test F1	test support	diagnosis
`GREETING`	0.79	44	fine
`FEAR_INDUCTION`	0.40	51	precision 0.93, recall **0.25** — too conservative
`BENIGN`	0.18	27	needs far more legitimate-call variety
`ISOLATION`	0.20	4	**starved** — 37 caller-side training examples
`AUTHORITY_CLAIM`	0.13	15	confused with `GREETING`
`VERIFICATION_DEMAND`	0.08	24	precision **1.00**, recall 0.04 — barely ever fires
`PAYMENT_SETUP`	0.00	2	**starved**
`PAYMENT_EXECUTION`	0.00	4	**starved**

Task: regenerate a larger corpus — target ~1,000+ calls with materially more archetypes, weighted hard toward `ISOLATION`, `VERIFICATION_DEMAND`, `PAYMENT_SETUP`, `PAYMENT_EXECUTION`, and a wider spread of `BENIGN`. Then rerun `train.py` + `eval_backends.py`.

The fix is more data, not more epochs. Do not tune hyperparameters against this; the split is telling the truth.

Two invariants must stay in lockstep or the served model silently underperforms: 1. the speaker-tagged `previous [SEP] current` join — `ml/train.py::render` **and** `MuRILStageClassifier.predict`; 2. label ordering, which travels inside the checkpoint config and is asserted on export.

> **Do not train on Apple MPS.** On torch 2.2 / transformers 4.44 a full run completed every step with loss pinned at exactly `ln(8) = 2.079` — the uniform-prior loss — and never left initialisation. CPU is the default for this reason.

Promotion is automatic once it wins: `load_classifier()` gates on `backend_comparison.json`, so a better model promotes itself. Force with `PRESAGE_CLASSIFIER=muril` to test.

P1 — Live audio / ASR (the biggest functional gap)

Nothing is wired. The **contract already anticipates it** — `partial_text` fields exist, `schema/models.py:336` says *"Audio rides as binary frames"*, and the `asr:local_fallback` degradation tag is defined — but:

- `routes/session.py::session_socket` is **`receive_json` only**. It does not accept binary frames. This is the concrete gap between contract and transport.
- No ASR implementation exists anywhere. `SARVAM_API_KEY` is in `.env.example` and referenced by nothing.
- `coercion.py` is written to consume pitch variance and pause ratio from ASR word timings, and currently runs on lexical features alone with `coercion:text_only` recorded and a lower cap.

Task: accept binary audio on the WS, run local `faster-whisper` for dev, swap in Sarvam for the live run, feed word timings into `CoercionTracker`, and emit `partial_text` on interim results. The engine side is already shaped for this — it's transport + an ASR adapter.

P1 — Guardian notification is in-app only

The state machine is complete and correct (`IDLE → ALERTING → ACKNOWLEDGED`, with the payment circuit breaker gated on it). But **no message actually leaves the process** — no SMS, no push, no call. The "alert" is a state field the `/guardian` screen polls.

Task: an outbound adapter (SMS/WhatsApp/push) fired on the `GUARDIAN_ALERTED` event, with the same explicit-degradation discipline as everything else — if it can't send, tag it, don't pretend.

P2 — Payment hold is simulated

`attempt_payment` / `cancel_payment` / `approve_payment` are a faithful in-session circuit breaker, but there is no UPI/bank integration. For the demo this is the right call; for anything real it's the hard part (and mostly a regulatory problem, not a code one). **Document it as simulated — don't imply otherwise in the pitch.**

P2 — LLM explainer is implemented but unconfigured

`llm.py` has working Gemini, Ollama, and Anthropic backends. `/api/health` currently reports `{"backend": "none", "configured": false}`, so explanations fall back to templates.

Task (5 minutes): set `PRESAGE_LLM=gemini` + `GEMINI_API_KEY` in `.env`, or run `scripts/ollama-up.sh` for the fully-offline path. Then confirm `/analyzer` returns a prose explanation and that it still **never touches a score**.

P2 — Dense retrieval not installed

Implemented and auto-selecting; just uncomment `sentence-transformers` in `services/api/requirements.txt`. Downloads ~90 MB on first run, which is why it's not a default. `degraded: ["rag:lexical"]` clears when it loads.

P3 — Smaller gaps

- **No OCR.** Screenshots must be typed out. Deliberate — a confident verdict on an empty string is worse than declining.
- **No persistence.** Sessions are in-memory and die with the process. Fine for a demo; a blocker for anything longitudinal.
- **No frontend tests.** Typecheck only. The backend has 16 regression cases; the UI has none. This is now the largest remaining gap in the check suite.
- **Real-world transfer unmeasured.** 100% synthetic training data. This is an honest limitation to state out loud, not a bug to hide.

Suggested split

These are independent — minimal merge conflict surface.

Track	Area	Why it's separable
A	P0 corpus + retraining	Touches `ml/` only. Nothing in `services/` or `apps/`
B	P1 audio/ASR + coercion prosody	Touches `routes/session.py` + a new adapter + `coercion.py`

Then whoever finishes first takes P1 guardian notifications and the CI action.

The one shared file to coordinate on is `schema/`. If you add a field, add it to `models.py` *and* `types.ts` in the same commit and run `check_contract.py`. A commit that changes one and not the other fails the check for whoever pulls next, and they'll think they broke it.

Before you push, always

```
.venv/bin/python -m pytest services/api/tests -q   # 16 passed
.venv/bin/python schema/check_contract.py          # contract consistent
npm run typecheck --prefix apps/web                # silent = clean
```

Note the `.venv/bin/python` on the contract check — bare `python3` fails with `ModuleNotFoundError: pydantic`.